Chapter 1

Machine learning for combinatorial optimization

1.1 Context and motivation[1]

Recent years have seen increased research interest in using models and methods from the machine learning (ML) community for solving combinatorial optimization problems (CO). In this chapter, we first discuss some key ideas and techniques found within this line of work. Based on this, the next chapter details our own version of this methodology for solving the crossing time problem introduced in Chapter **??**.

In typical CO problems, we try to find some the best option from some set of discrete structures such as graphs, sets, sequences, etc. Typical problems involve, for example, finding some path or route through a graph, like in the traveling salesman problem; selecting some optimal subset, like in the knapsack problem or set cover problem; or selecting the best sequence in which machines are assigned to pending jobs, as in job-shop scheduling. Generally speaking, we can think of CO problems as being concerned with finding an optimal configuration vector $v \in V$ from a finite[2] set of candidates, such that some objective function $f : V \to \mathbb{R}$ is minimized. Since there are only finitely many candidates, a straightforward solution is to just evaluate the objective for each candidates $v \in V$ and then output some configuration for which $f(v)$ is minimal. However, for all but trivial problems, this is simply not feasible in any reasonable amount of time, which is the central motivation in the field of combinatorial optimization.

In almost all practically relevant CO problems, the *objective* function f exhibits some kind of regularity. Therefore, when developing algorithms, a good strategy is to look for such structure in the problem that can be exploited algorithmically. Such structural insights can be precise, like in Theorem **??**, and methods like those based on the branch-and-bound scheme try to take advantage of such well-defined structure. Often, such algorithms come with some kind of guarantee on the quality of the returned solution. Other algorithms are based on some hard-to-define intuition about what optimal solutions should look like. This very much applies to so-called *local search* algorithms.

Literature overview. Within this context, two main motivations for applying ML for solving CO problems can be identified in the current literature [2], which we discuss next. The key publications on which the following discussion is based are collected in Table 1.1.

[1] The narrative of this section has been heavily influenced by the presentation in [2] and we have borrowed some of their notation to explain the algorithm optimization problem over problem distributions.

[2] This assumption simplifies our current discussion, but note that some typical combinatorial problems have infinitely many feasible solutions. It is not trivial to distinguish between *continuous* optimization and *combinatorial* optimization. We could call a problem combinatorial if at least one component v_i of the configuration $v \in \mathbb{R}^n$ is restricted to lie in some countable domain. For example, this is the case for mixed-integer programs, where some variables are restriced to the integers. However, some problems that should be called continuous according to this definition are more combinatorial in nature than it seems at first sight. For example, the feasible set of a linear program is generally not a finite set, but Dantzig's simplex algorithm shows that the optimal solution must be one of the finitely many vertices of the convex polytope.

Reference	Problem focus	Contribution / Method
Bengio et al. [2]	General CO problems	Survey on using ML for CO, framing the learning problem
Mazyavkina et al. [8]	General CO problems	Survey on using ML for CO, with a focus on reinforcement learning
Dai et al. [3]	Graph problems	Deep RL with graph embeddings to construct solutions
Vinyals et al. [11]	TSP	Pointer networks for sequence-based combinatorial problems
Kool et al. [7]	Routing problems	Attention model trained with RL for solving TSP, VRP
Gasse et al. [4]	MILP	Approximate strong branching policy with GNNs in branch-and-bound
Tang et al. [10]	ILP	Attention-based model for cutting plane selection trained with RL
Graves et al. [5]	General computation	Neural turing machine as a differentiable model of universal computation

Table 1.1: Some key references on machine learning for combinatorial optimization.

1. Fast approximations. We might want to replace certain expensive computations of an existing algorithms such as branch-and-bound with fast ML approximations. Please note that the approximate nature of ML models does not necessarily impact optimality guarantees of the original algorithm. This is clearly illustrated by [4], where the branching decision in branch-and-bound is approximated, and the line of work related to [10], where ML models are employed for cutting plane selection. In both examples, the algorithm is still guaranteed to obtain an optimal solution, as long as the model provides valid branching decisions or valid cutting planes. Schemes in which ML models are integrated within some existing CO algorithm have been referred to as *joint* methods.

2. Algorithm discovery. Instead of trying to improve existing algorithms, we could try to employ ML to explore the space of algorithms to find new, hopefully better, algorithms. This type of approaches can also be understood as constructing a solution from scratch or in terms of predicting the optimal solution, given the problem instance. Our method, that we will propose in **??**, belongs to this class of so-called *principal* methods.

Central ideas. In both cases, the premise is that *ML can be used to discover certain regularities of the problem automatically.* Therefore, one of the central ideas is to treat problem instances and their solutions and training data for a machine learning model [2]. This training data is used to fit a machine learning model and the hope is that it somehow captures model structure similar to our intuition. In the next section, we will make this notion more precise by presenting the process of designing algorithms for CO problems as an optimization problem in and of itself. After that, we show how to state this in terms policy optimization in some Markov decision process (MDP), opening up the application of optimization methods from reinforcement learning.

1.1.1 Optimizing the algorithm

Let $\omega = (V, f) \in \mathcal{I}$ denote some particular instance of a combinatorial problem and let $\pi \in \Pi$ denote some set of possible algorithm for solving it—we will make the definition of Π more precise later. Let $m : \mathcal{I} \times \Pi \to \mathbb{R}$ denote some function that measures the performance of executing algorithm π on problem instance ω. Function m may involve aspects like whether the algorithm actually finds a valid solution, optimality of the found solution, running time and usage of other resources like memory, etc. Using this notation, the goal of the algorithm developer can be succintly formulated as

$$\min_{\pi \in \Pi} m(\omega, \pi). \tag{1.1}$$

However, this single-instance situation is not very typical. More often we have some class of instances $\mathcal{I}$ of interest, say, the set of all traveling salesman problems. Let $m(\omega, \pi)$ measure the running time of the algorithm, then from the perspective of the classical worst-case running time analysis, the algorithm developer's goal is now

$$\min_{\pi \in \Pi} \max_{\omega \in \mathcal{I}} m(\omega, \pi). \tag{1.2}$$

Problem distribution. In principle, we wish to devise algorithms that perform as good as possible for all possible instances $\mathcal{I}$. However, in most practical application, we are only dealing with a narrow subset of $\mathcal{I}$, often consiting of very similar problems. To model the relative occurence and/or importance of problems, we can induce a probability distribution P over $\mathcal{I}$. To illustrate this, imagine some delivery company in Eindhoven that needs to solve many instances of the vehicle routing problem (VRP) every day [2]. The problems they encounter are often similar in some sense: many customers are located in the north, there is one large customer in the south, only a few regular customers are located at the TU/e campus; it often turns out to be beneficial to plan routes that use the ring road, etc. The company does not care about solving all possible VRPs, just their particular kind. This example is further illustrated in Figure 1.1.

There is some underlying, but unknown, probability distribution P describing the instances the company will likely face. This distribution captures the problem instances the company actually cares about. Hence, instead of (1.2), the company is actually more interested in

The free lunch theorem also motivates this sharper definition. *[1]*

$$\min_{\pi \in \Pi} \mathbb{E}_{\omega \sim P} \left[m(\omega, \pi) \right]. \tag{1.3}$$

However, since P is generally inaccessible, we need to resort to using samples, which could be obtained from historical company data, for example. Suppose that we have some set of training instances $D_{\text{train}} \sim P$ at our disposal, then we could approximate (1.3) by solving

$$\min_{\pi \in \Pi} \sum_{\omega \in D_{\text{train}}} \frac{1}{|D_{\text{train}}|} m(\omega, \pi). \tag{1.4}$$

Class of algorithms Π. To make the algorithm optimization idea outlined above more concrete, we first need to fix a suitable representation for the set of algorithms Π. In principle, we could take Π to consist of all possible C programs, or all Python programs, for example. However, real-world code runs on a complex stack of compilers, interpreters, and other complicating factors like hardware optimizations. Therefore, it might be more desirable to work with simplified models of computation. A common modeling choice in the ML for CO literature is to represent algorithms as policies for Markov decision processes (MDPs). This perspective naturally connects algorithm optimization to reinforcement learning methods, which many existing works indeed leverage [3, 7, 8]. In what follows, we show how the MDP framework arises naturally as a model of computation and explain why it provides a particularly useful abstraction within the contex of algorithm optimization.

1.1.2 A model of computation

A key idea in the existing ML for CO literature is to model the class of candidate algorithms in terms of policies for some Markov decision process (MDP).

Finite-state automaton. First, we show how the execution of an algorithm can be viewed as a particular state transition sequence of some deterministc finite-state automaton (DFA). Let $\mathcal{S}$ be the finite set of states, with $\mathcal{S}_0 \subseteq \mathcal{S}$ denoting the initial states, encoding problem instances, and $\mathcal{S}_f \subseteq \mathcal{S}$ denoting the final states that represent possible outputs. The remaining states capture the *internal state* of the algorithm, consisting of all relevant data structures in memory. Let $\mathcal{A}$ denote the finite non-empty set of *actions*[3] and let $p : \mathcal{S} \times \mathcal{A} \to \mathcal{S}$ be some *transition function* that specifies the next state when a particular action is chosen. In this context, an algorithm can be viewed as a *policy* function $\pi : \mathcal{S} \to \mathcal{A}$ that provides the action to be taken from each state. The policy parameterizes the automaton's execution similar to how program code determines the state transitions of a computer. In this model,

[3]Note that "input alphabet" is actually more common terminology for DFAs, because of their origins in complexity theory and formal languages, where they are usually defined in terms of *accepting* or *rejecting* a certain sequence of input symbols depending on the final state. In some sense, they are strictly less powerful than Turing machines; the latter is generally regarded as the most powerful/complete model of computation (Church-Turing thesis).

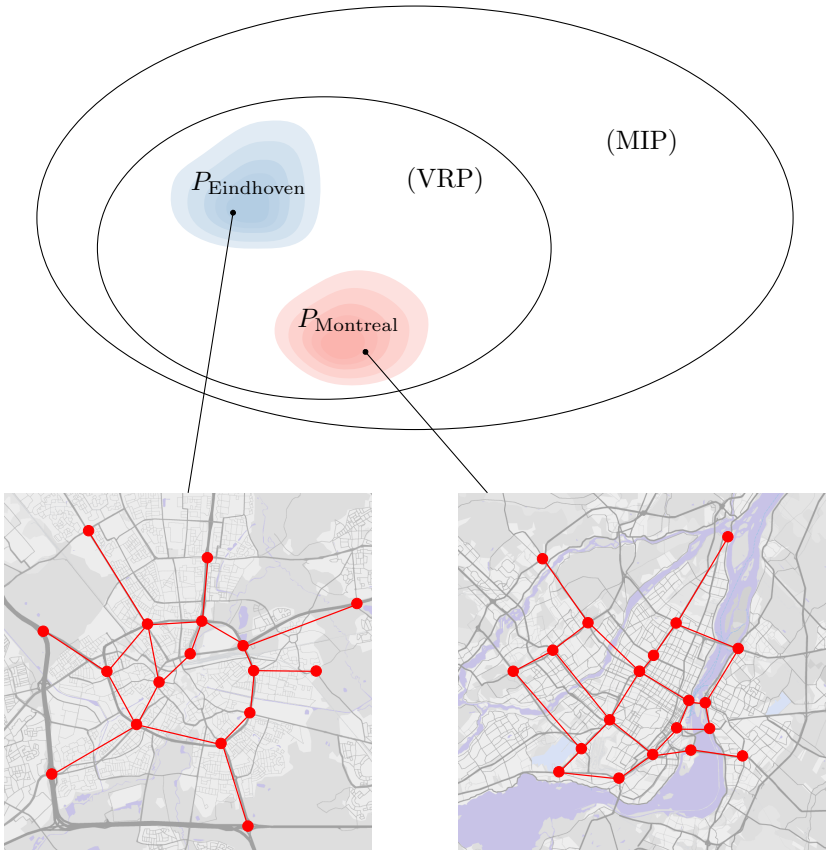

Figure 1.1: *Illustration of how problems occuring in practice have a naturally narrow distribution within the full space of problem.* The top of the figure illustrates a typical hierarchical relationship between classes of combinatorial problems: instances of the vehicle routing problem (VRP) can be understood as specific instances of mixed-integer programs (MIP) and this relationship is mathematically well-defined. In contrast to this clear boundary between problems, it might make more sense to consider a less sharp boundary between classes of problems by considering their distribution instead. For example, we might consider the distribution $P_{\text{Eindhoven}}$ of VRPs encountered by some imaginary delivery company in Eindhoven. The left graph illustrates what a typical sample from this distribution could look like. Suppose that the delivery company also has an overseas branch in Montreal, then the distribution P_{Montreal} of problems faced by that branch will be very different. For example, the Dommel river does not have such a large influence on the connectivity in Eindhoven compared to that of the St. Lawrence River. Furthermore, Eindhoven has some sort of circular connectivity due to the ring road, whereas large parts of Montreal are fairly Manhattan-like. It should be clear that statements like these are very cumbersome to formalize precisely, which is one of the main motivations for exploring how machine learning can be applied to such problem settings.

the performance of running algorithm π on instance ω is measured[4] in terms of the unique final state s_f^π by some cost function $c : \mathcal{S}_f \to \mathbb{R}$, such that we have $m(\omega, \pi) = c(s_f^\pi)$.

Designing the automaton. With this model in hand, we can now define Π to be the set of valid policy functions for some fixed automaton. Hence, the automaton definition becomes a key decision in the algorithm optimization, because fixing a particular state space effectively restricts the class of algorithms. The algorithm designer can choose to use a very general automaton, similar to a Turing machine, or incorporate prior knowledge and intuition by carefully defining a more restricted state space. To illustrate this, suppose we let the state space represent all possible permutations of nodes in a traveling salesman problem, then Π would naturally model all possible local search algorithms. Indeed, any permutation of the nodes corresponds to a valid tour such that any state is a valid solution, so we have $\mathcal{S}_f = \mathcal{S}$. The transition function p implicitly defines the *neighborhood* of each solution, and the policy π specifies which neighbor to choose, capturing the essence of the local search process.

Markov decision process. We can generalize the automaton defined above as follows. Instead of a single next state, the transition function p becomes a mapping to distributions of states. similarly, we let policies map to distribution over actions. Therefore, we now write

$$p : \mathcal{S} \times \mathcal{A} \to \Delta(\mathcal{S}) \quad \text{and} \quad \pi : \mathcal{S} \to \Delta(\mathcal{A}).$$

This generalization turns the automaton into a finite-horizon MDP, with $r = -c$ taking the role of final reward function. To take into account the additional stochasticity this introduces, we can generalize the algorithm development objective (1.3) to

$$\max_{\pi \in \Pi} \mathbb{E}_{\omega \sim P} \left[\mathbb{E}_\pi \left[r(s_f^\pi) \mid \omega \right] \right], \tag{1.5}$$

where the inner expectation corresponds to the randomness of policy evaluation in the MDP, because s_f^π is now a random variable, and Π still denotes the set of all **deterministic** policies.

Why is stochasticity useful? It might not be immediately clear why we would want to introduce stochasticiy in a seemingly deterministic problem environment. We will discuss two reasons why the MDP framework is nevertheless an attractive starting point.

First of all, non-deterministic transitions can be used to model certain parts of the computation that are too complex to model explicitly. This has for example been done in previouw joint methods, in which some part of an existing CO solver is augmented by replacing a specific component with a ML model. For example, we might take some existing integer programming solver as the base algorithm, while parameterizing a specific component, say, cutting plane selection, as done in [10]. In that case, the actions $\mathcal{A}$ correspond to selecting some valid cutting plane, upon which the state transition p models the whole of the execution of the solver until the next time a cutting plane has to be selected.

Although the final policy should be deterministic, during the optimization of (1.5), stochastic policies might be considered. The reasons for this are mainly of a technical nature, for example to promote exploration or to make sure the policy gradient exists. Alternatively, we could interpret action probabilities as modeling the uncertainty about which actions are optimal. In some sense, this is similar to the practice of considering *relaxations* when solving integer programs. In the relaxed version of an integer program, the integer constraints are removed, which makes it much easier to solve. Of course, the final solution must satisfy the integer constraints, so we need some sort of *rounding procedure* to return to integer values. Similarly, we eventually want to find an optimal deterministic policy, but we relax the "determinism constraint" by allowing distributions of actions instead. We need to specify some "rounding procedure" to return to deterministic (also called *degenerate*) distributions, for example, by performing greedy action selection, or by other search procedures proposed like *active search* [1] or *beam search* [6].

[4] To accomodate this, states may need to carry auxiliary meta-information such as total number of transitions or bounds on the quality of the solution. Actual running time is not captured by this simple model formulation, but the number of transitions can be used as a proxy. This is no immediate problem: the current model is merely used as a way of introducing the general methodology.

***Computing MDP* framework.** We now summarize the algorithm optimization paradigm, outlined above, by listing the main components of the MDP and their modeling role:

- The **state space** models all configurations of the *internal state* of the algorithm. These may correspond to hand-crafted, problem-specific data structures or more model actual computer memory more generally.
- The set of **actions** may be thought of as the *instruction set*.
- The **state transitions** determine how these instructions are executed. Stochastic state transition can be used to model fixed parts of existing algorithm that are too complex and expensive to consider in their entirety.
- The **reward function** measures how well an algorithm performs when executed on a particular problem instance.
- The goal is to find a **deterministic policy** that maximizes (1.5). Stochastic policies are considered during optimization for technical reasons.

Remark. *Within the above framework, we can also think about the analogy between procedures, in computer code, and the* options framework *[9], which is a classical approach of temporal abstraction in MDPs.*

Remark. *A closely related line of research that takes the above paradigm to the extreme is the study of* Neural Turing Machines *[5], where neural networks are used to parameterize the execution of a Turing machine, which is in some sense a universal and complete model of computation. It has been demonstrated that simple algorithms for tasks such as sorting or information retrieval can, in principle, be "learned from experience" in this way. While this represents a conceptually powerful idea, in practice such models are notoriously difficult to train. The vast space of possible policies effectively imposes only a very weak prior. Consequently, it is often more practical to design parameterizations that are better aligned with specific problem structures, thereby improving convergence. In this light, our own MDP formulation of* **??**, *can be understood as introducing a particularly strong prior.*

Remark. *Regarding the use of probabilities in the model, note that the transition probabilities in the MDP do not necessarily represent any inherent randomness in the system. The underlying computational process is, in principle, fully observable and deterministic, meaning that the exact state can always be known. Therefore, there is no* aleatoric uncertainty *associated with the system. However, as discussed above, it can still be useful to represent certain components of the computation probabilistically. In this case, we are modeling* epistemic uncertainty*—that is, uncertainty arising from incomplete knowledge or imprecise information, which could, in principle, be reduced by gaining more data or understanding.*

1.2 Overview of methodology

We argued that the algorithm optimization problem sketched above can be naturally stated in terms of finding an optimal policy in some MDP. This enables us to draw from the vast reinforcement learning literature for policy optimization methods. We outline a general methodology, centered around the follow three aspects. First, we explain that (i) generalization should be regarded a key objective. Obtaining policies that generalize well also justify the typically long time needed for traning, which can happen offline. Next, we discuss two important engineering-type aspects, which are (ii) the way in which the policy is parameterized and (iii) the corresponding optimization procedure.

Performance evaluation. Performance evaluation is mainly based on expected solution quality and on expected running time. Return to the issue of measuring running time using the MDP model.

1.2.1 Generalization

One of the main challenges in ML for CO is obtaining algorithms that generalize to instances not seen during training. This means that the learned policy should not only perform well on the training data set but also remain robust when facing instances from P not seen before in D_{train}. For this reason, it makes sense to use a set of validation samples D_{valid} to gauge how well the learned policy generalizes to unseen instances.

Because we are dealing with sequential decision making models, we emphasize that generalization also applies to *unseen internal states*. We want policies that do not break down immediately when slightly different internal states are encountered—for example, when some decision early on leads to a novel configuration of the remaining algorithmic state that was rarely, if ever, observed during training. It has been observed that policy optimization with RL techniques, compared those based on the imitation learning principle, which we will discuss in more detail below, can help improve generalization [1]. This is not surprising, since RL methods are specifically designed to explore the state space actively. Unlike imitation learning, which is constrained by the trajectories of an expert, RL encourages the discovery of alternative strategies and adaptation to unseen states, which tends to foster robustness.

There are two important axes to consider when measuring generalization capabilities: instance *size* and instance *structure*. For example, in the traveling salesman problem (TSP), the number of nodes is a natural measure of size: scaling from 20 nodes to 100 nodes dramatically increases the size of the solution space. A well-generalizing policy should gracefully adapt to scaling up or down, even if it has only been trained on a narrow size range. The *structure* axis, on the other hand, refers to the occurence of patterns or lack of them within problem instances. For TSP, this might include differences between uniformly random point clouds, clustered distributions of nodes, or instances derived from real-world city layouts. Policies that generalize well with respect to structural differences should be able to transfer knowledge about synthetic training data to instances encountered in the real-world.

In practice, strong generalization requires balancing both axes: a policy that scales to larger sizes but fails to adapt to structural changes is just as limited as one that can handle different structures but collapses under size increases.

1.2.2 Policy parameterization

Why parameterize at all? Because it is not guaranteed that the policy space is finite. In the context of CO problems, we can safely assume that the state and action space are discrete (countable). In principle, the deterministic policy $\pi : \mathcal{S} \to \mathcal{A}$ or stochastic policy $\pi : \mathcal{S} \to \Delta(\mathcal{A})$ can thus be represented by a look-up table, mapping each state to an action or action distribution, respectively. However, such a tabular method becomes intractable very quickly, because the size of the state and action space typically explodes for any practically relevant problem. For this reason, it is common to compress the state-action mapping into some lower-dimensional parameter vector. Instead of directly optimizing over some class of

algorithms Π, we try to optimize some vector $\theta \in \mathbb{R}^n$ that parameterizes the policy π_θ. In this case, the algorithm optimization problem with expected cost can then be written as

$$\min_\theta \mathbb{E}^{\pi}_{\omega \sim P} \left[m(\omega, \pi_\theta) \right]. \tag{1.6}$$

An additional benefit of such parameterization, is that it enables policies to generalize smoothly to unseen states. Furthermore, it allows the algorithm developer to incorporate further prior knowledge of the problem at hand, for example, by designing the parameterization such that it is insensitive to certain invariances among states, think for example of permutation invariance of the nodes when working with graph-based problems.

The appeal of the ML for CO paradigm lies in the idea that high-capacity neural network-based models can capture patterns in the problem. In this case, the parameter represents the weights and biases of the neural network. Although the universal approximation abilities of neural networks are very appealing, their use raises some issues in the context of CO problems. For example, CO problems often involve unordered sets instead of ordered sequences. Therefore, the neural network should be invariant to the ordering of the elements in the input. One early method to deal with this issue was the pointer network [11]. Later, more general attention mechanisms like the transformer architecture have become more mainstream [7].

1.2.3 Policy optimization

We can identify two main learning paradigms for finding good algorithms π, which also have natural ties to the two main motivations discussed above. Suppose that we have some heavy computation $f(\cdot)$ which we want to replace with a fast approximation $\hat{f}(\cdot)$. A natural strategy is to collect samples of $f(x)$ for a set of inputs $x \in X$ and then consider the supervised learning problem of minimizing some loss function $\ell(f(x), \hat{f}(x))$. We could say that this method is *learning from demonstration*. It could happen that policies that are not close to the expert perform well nevertheless, because there exists some alternative good strategy.

Alternatively, we could try to search the space of possible algorithms more directly, by trying to optimize the expected reward. The methods typically used here come from the reinforcement learning community [8]. Recall the motivation for a continuous parameterization of the policy π using some parameter vector θ. Apart from the benefits stated above, it enables us a to employ a class of methods known as *policy gradient* methods. These methods are based on the *policy gradient theorem*, which essentially provide an efficient way to calculate the impact of a slight change of the parameter on the final reward. This means that the policy optimization problem can be tackled using gradient-descent.

Beam search can be understood as searching over execution trajectories of algorithm actions parameterized by probability distributions.

In some sense, the beam search procedure itself can also be regarded as just another algorithmic prior on a higher level of abstraction. From this perspective, we could even parameterize some parts of the beam search procedure itself and subject this to learning. Think for example of beam width.

Bibliography

[1] Irwan Bello et al. *Neural Combinatorial Optimization with Reinforcement Learning*. Jan. 2017. arXiv: 1611.09940.

[2] Yoshua Bengio, Andrea Lodi, and Antoine Prouvost. *Machine Learning for Combinatorial Optimization: A Methodological Tour d'Horizon*. Mar. 2020. arXiv: 1811.06128 [cs, stat].

[3] Hanjun Dai et al. *Learning Combinatorial Optimization Algorithms over Graphs*. Feb. 2018. arXiv: 1704.01665 [cs].

[4] Maxime Gasse et al. *Exact Combinatorial Optimization with Graph Convolutional Neural Networks*. Oct. 2019. arXiv: 1906.01629 [cs].

[5] Alex Graves, Greg Wayne, and Ivo Danihelka. *Neural Turing Machines*. Dec. 2014. arXiv: 1410.5401 [cs].

[6] Wouter Kool. "Learning and Optimization in Combinatorial Spaces". PhD thesis. 2022.

[7] Wouter Kool, Herke van Hoof, and Max Welling. *Attention, Learn to Solve Routing Problems!* Feb. 2019. arXiv: 1803.08475 [stat].

[8] Nina Mazyavkina et al. *Reinforcement Learning for Combinatorial Optimization: A Survey*. Dec. 2020. arXiv: 2003.03600.

[9] Richard S. Sutton, Doina Precup, and Satinder Singh. "Between MDPs and Semi-MDPs: A Framework for Temporal Abstraction in Reinforcement Learning". In: *Artificial Intelligence* 112.1-2 (Aug. 1999), pp. 181–211.

[10] Yunhao Tang, Shipra Agrawal, and Yuri Faenza. *Reinforcement Learning for Integer Programming: Learning to Cut*. July 2020. arXiv: 1906.04859 [cs, math, stat].

[11] Oriol Vinyals, Meire Fortunato, and Navdeep Jaitly. *Pointer Networks*. Jan. 2017. arXiv: 1506.03134 [cs, stat].